

PHILIXA 6.0

V1-MVP Design - Commitment and Memory Copilot

Version: V1-MVP Fixed Implementation Spec Scope: Backend-first production-style MVP Primary user: Banking Relationship Manager (RM)

1. Role and Build Discipline

Build only the V1-MVP of PHILIXA 6.0.

The system must behave like a production SaaS backend, not a student demo. It must be complete enough to demonstrate an end-to-end RM workflow through FastAPI, Swagger, SQLite, SQLAlchemy, Pydantic, tests, Docker, logging, and clear environment configuration.

Do not build roadmap features from V1.5, V1.9, V2, or V3. Do not create empty modules, placeholder services, or future-feature stubs.

2. V1 Scope Contract

V1-MVP is a paste-notes workflow only.

The RM does one primary action:

- Paste raw meeting notes into PHILIXA.

PHILIXA then automatically:

- Creates a meeting summary.
- Identifies or requests confirmation for the client.
- Detects key discussion points.
- Detects concerns.
- Extracts commitments.
- Extracts due dates only when reliable.
- Deduplicates active commitments.
- Updates client memory.
- Returns instant client context.

Out of scope for V1:

- Voice recording.
- Speech-to-text.
- Whisper integration.
- Reminder scheduler.
- Escalation workflow.
- Relationship risk engine.
- Manager dashboard.
- AI coaching or roleplay.
- Revenue intelligence.

- Product recommendations.
- Bank statement analysis.
- Cross-sell or upsell prediction.

Important wording rule:

V1 does not promise automated reminders. V1 promises that commitments are captured, stored, deduplicated, and surfaced clearly whenever the RM opens a client profile or processes a new meeting note.

3. Core Problem

Relationship Managers handle many client interactions. During calls and meetings they make promises such as:

- "I will send documents tomorrow."
- "I will call you next week."
- "I will provide the loan approval status."
- "I will share investment options."

After many meetings, these commitments are often forgotten or scattered across notes.

This causes:

- Customer dissatisfaction.
- Trust loss.
- Relationship damage.
- Revenue leakage.
- Internal escalations.

Mission:

Capture and surface every important client commitment so the RM always knows what happened, what is pending, and what matters before the next interaction.

4. Ideal RM Workflow

Step 1: RM finishes a client meeting. Step 2: RM opens PHILIXA. Step 3: RM pastes raw meeting notes. Step 4: PHILIXA processes the note through the AI extraction layer. Step 5: If client identity is clear, PHILIXA updates memory automatically. Step 6: If client identity is unclear, PHILIXA stores the meeting as "Client Identification Required" and asks for client confirmation. Step 7: RM can open the client profile and see instant context in less than 5 seconds.

Target effort:

- Normal case: under 30 seconds.
- Ambiguous case: one lightweight confirmation step.

The product principle is AI-first, confirmation-only. The RM should not manually maintain CRM-style forms.

5. Example Input and Output

Example raw meeting note:

Met Rajesh Sharma today.
Interested in business loan.
Concerned about processing time.

Promised documents by Friday.
Asked for approval status update in 3 days.

Expected AI output:

```
{
  "client_identification": {
    "status": "identified",
    "matched_client_id": null,
    "suggested_client_name": "Rajesh Sharma",
    "confidence": 0.92,
    "requires_confirmation": false
  },
  "meeting_summary": "Rajesh Sharma discussed interest in a business loan and raised concern about processing time.",
  "key_discussion_points": [
    "Interested in business loan",
    "Concerned about loan processing time"
  ],
  "concerns": [
    {
      "description": "Processing time concern",
      "severity": "medium",
      "confidence": 0.86
    }
  ],
  "commitments": [
    {
      "description": "Send documents",
      "owner": "RM",
      "due_date_text": "by Friday",
      "due_date": "YYYY-MM-DD",
      "due_date_confidence": 0.82,
      "status": "pending",
      "confidence": 0.9
    },
    {
      "description": "Provide approval status update",
      "owner": "RM",
      "due_date_text": "in 3 days",
      "due_date": "YYYY-MM-DD",
      "due_date_confidence": 0.88,
      "status": "pending",
      "confidence": 0.9
    }
  ],
  "action_items": [
    "Send documents",
    "Provide approval status update"
  ]
}
```

The final dates must be calculated using the meeting date or processing date. If the date cannot be reliably inferred, store null and keep the original due_date_text.

6. Client Identification Rules

Every processed meeting must be attached to:

- An existing client.
- A newly created client.
- Or an unresolved meeting record requiring client confirmation.

Rules:

- If a client name is explicitly mentioned and confidently matches an existing client, attach the meeting to that client.
- If a client name is explicitly mentioned but no existing client matches, create a new client profile.
- If the note does not clearly identify the client, do not guess.
- If confidence is below threshold, set status to "client_identification_required".

Recommended threshold:

- Auto-match existing client when confidence ≥ 0.85 .
- Auto-create new client when explicit name confidence ≥ 0.80 and no close match exists.
- Require confirmation when confidence < 0.80 or when multiple close matches exist.

Examples:

- "Met Rajesh Sharma today." -> client can be identified or created.
- "Customer interested in home loan. Wants callback Friday." -> client identification required.

7. Due Date Rules

AI must extract due dates only when they are explicitly stated or can be reliably inferred.

Allowed:

- "tomorrow"
- "next Monday"
- "by Friday"
- "in 3 days"
- exact calendar dates

Not allowed:

- "soon"
- "later"
- "sometime next week"
- "when possible"

If due date is vague, missing, or low confidence:

- Store due_date as null.
- Store original due_date_text.
- Store due_date_confidence below threshold.
- Do not invent a date.

Recommended threshold:

- Store calculated due_date only when due_date_confidence ≥ 0.75 .
- Otherwise store due_date = null.

8. Commitment Extraction Rules

A commitment is an action that someone is expected to perform after the meeting.

Minimum fields:

- id
- client_id
- meeting_id
- description
- owner
- status

- due_date
- due_date_text
- extraction_confidence
- created_at
- updated_at

Allowed statuses:

- pending
- completed

V1 status updates must stay lightweight:

- RM can mark a pending commitment as completed.
- RM can reopen a completed commitment as pending.

No reminder scheduling or escalation is included in V1.

9. Commitment Deduplication Rules

Before creating a new active commitment, PHILIXA must check for similar pending commitments for the same client.

If a matching pending commitment already exists:

- Update the existing commitment.
- Refresh due date if the new note provides clearer or newer information.
- Append the new meeting reference.
- Do not create a duplicate active commitment.

Deduplication comparison should consider:

- Same client.
- Same or similar action phrase.
- Same owner.
- Existing status is pending.
- Semantic similarity or normalized text similarity.

Example:

Meeting 1: "Send documents Friday." Meeting 2: "Reminder: still need to send documents." Result: one pending commitment, not two.

Implementation rule:

Use a simple deterministic fallback first, such as normalized lowercase text matching plus token overlap. If the AI layer is available, it may also return a `duplicate_of_commitment_id` candidate. The backend must still validate that the duplicate belongs to the same client and is pending.

10. Client Memory Management

Purpose:

Create a living memory of each client without overwhelming the RM.

Client profile should show:

- Client name.
- Last meeting summary.
- Pending commitments.
- Major concerns.
- Recent relationship notes.
- Rolling memory summary.

Meeting history must be retained in storage.

As meeting volume grows:

- Generate rolling summaries.
- Consolidate recurring concerns.
- Highlight active commitments.
- Surface only recent and actionable context in the profile.
- Keep older raw meeting records available in history.

Memory compression rule:

After each processed meeting, update a concise rolling_summary field for the client. The summary should preserve durable facts, recurring concerns, and important context while avoiding a long chronological dump.

11. API Contract

Base path:

/api/v1

Required endpoints:

POST /meeting-notes/process

Input: raw meeting notes, optional meeting_date, optional known_client_id. Output: AI extraction result, saved meeting record, client status, created or updated commitments.

POST /meeting-notes/{meeting_id}/confirm-client

Input: selected client_id or new client name. Output: meeting attached to client and memory updated.

GET /clients

Output: list clients with basic context counts.

GET /clients/{client_id}/memory

Output: instant RM briefing with last meeting summary, pending commitments, concerns, rolling summary, and recent notes.

GET /clients/{client_id}/meetings

Output: meeting history for a client.

GET /commitments

Filters: status, client_id, due_before. Output: commitments matching filters.

PATCH /commitments/{commitment_id}/status

Input: pending or completed. Output: updated commitment.

GET /health

Output: service health, database connectivity, app version.

Swagger documentation must show clear request and response examples for the main workflow.

12. Database Contract

Use SQLite for V1-MVP and SQLAlchemy ORM.

Required tables:

clients

- id
- name
- normalized_name
- rolling_summary
- relationship_notes
- created_at
- updated_at

meetings

- id
- client_id nullable
- raw_notes
- meeting_date
- summary
- key_discussion_points_json
- concerns_json
- status
- client_identification_status
- client_identification_confidence
- created_at
- updated_at

Allowed meeting statuses:

- processed
- client_identification_required

commitments

- id
- client_id
- description
- normalized_description
- owner
- due_date nullable
- due_date_text nullable
- due_date_confidence
- status
- extraction_confidence

- created_at
- updated_at

commitment_meeting_links

- id
- commitment_id
- meeting_id
- created_at

ai_extraction_logs

- id
- meeting_id nullable
- provider
- model
- prompt_version
- raw_response_json
- parsed_response_json
- success
- error_message nullable
- created_at

Store raw AI responses for debugging, but do not log sensitive customer notes into normal application logs.

13. Pydantic Schema Contract

Core request:

```
{
  "raw_notes": "string",
  "meeting_date": "YYYY-MM-DD",
  "known_client_id": 1
}
```

Core response:

```
{
  "meeting_id": 1,
  "client_status": "identified",
  "client_id": 1,
  "requires_client_confirmation": false,
  "meeting_summary": "string",
  "commitments_created": [],
  "commitments_updated": [],
  "pending_commitments": [],
  "warnings": []
}
```

Client memory response:

```
{
  "client_id": 1,
  "client_name": "Rajesh Sharma",
  "last_meeting_summary": "string",
  "pending_commitments": [],
  "major_concerns": [],
  "recent_relationship_notes": [],
  "rolling_summary": "string"
}
```


14. AI Layer Contract

Supported providers:

- Gemini.
- Groq.

Provider is selected through environment variables.

Required behavior:

- Return strict JSON.
- Never invent due dates.
- Include confidence scores.
- Include client identification status.
- Include due_date_text separately from calculated due_date.
- Include warnings when information is ambiguous.

If AI provider fails:

- Return a controlled API error.
- Do not write partial or misleading records unless the failure happens after a validated extraction.
- Log technical error details without exposing sensitive meeting notes in normal logs.

Prompt versioning:

- Every extraction must store prompt_version in ai_extraction_logs.
- This keeps the MVP debuggable when prompts change.

15. Security and Privacy Contract

Even as an MVP, PHILIXA handles banking-style client data.

Required:

- Use environment variables for API keys and provider config.
- Never hardcode AI keys.
- Do not print raw meeting notes in normal logs.
- Add basic API key authentication for protected endpoints.
- Keep /health public or minimally safe.
- Validate request sizes to avoid huge note uploads.
- Return safe error messages to API clients.
- Keep .env out of git.

Recommended V1 limits:

- raw_notes max length: 10000 characters.
- client name max length: 120 characters.
- commitment description max length: 500 characters.

16. Architecture Contract

Suggested structure:

```
app/  
  main.py  
  api/  
    v1/  
      routes_meeting_notes.py  
      routes_clients.py  
      routes_commitments.py  
      routes_health.py  
  core/  
    config.py  
    logging.py  
    security.py  
  database/  
    session.py  
    base.py  
  models/  
    client.py  
    meeting.py  
    commitment.py  
    ai_extraction_log.py  
  schemas/  
    meeting_note.py  
    client.py  
    commitment.py  
  services/  
    meeting_processing_service.py  
    client_identification_service.py  
    commitment_service.py  
    memory_service.py  
  ai/  
    provider.py  
    prompts.py  
    parser.py  
  utils/  
    text_normalization.py  
  tests/
```

Do not create folders for out-of-scope future systems.

17. Testing Contract

Required tests:

- Process notes with clear client name and commitments.
- Process notes with unknown client and require confirmation.
- Confirm client for unresolved meeting.
- Extract explicit due date.
- Avoid invented due date for vague phrases.
- Deduplicate matching pending commitments.
- Mark commitment completed.
- Fetch client memory under normal conditions.
- AI provider failure returns controlled error.
- API key protection works for protected endpoints.

Tests may mock the AI provider. The MVP should not depend on live AI calls for unit tests.

18. Definition of Done

The V1-MVP is successful when:

- RM can paste meeting notes through Swagger.
- System extracts structured meeting intelligence.
- Clear client notes attach automatically.
- Ambiguous client notes require confirmation instead of guessing.
- Commitments are stored with status and due date rules.
- Duplicate pending commitments are avoided.
- Client memory updates after every confirmed meeting.
- RM can open client memory and get context in under 5 seconds.
- The project runs locally with documented commands.
- The project has tests, Docker support, README, logging, error handling, and environment variable configuration.

Final V1 promise:

PHILIXA V1-MVP captures and organizes client meeting memory with minimal RM effort. It does not yet automate reminders, voice processing, manager visibility, coaching, or revenue intelligence.